

Final Report Document
SP-14 - Green - Chess AI
CS 4850 - Section 03 – Fall 2025
Advisor: Sharon Perry
Dec 1, 2025

**Team Members: Matt Staats, Carlos Pena, Jason Nguyen,
Chiagoziem Iteogu**

GitHub Link: [SP-14-Green-Chess-AI/SP-14-Green-Chess-AI](https://github.com/SP-14-Green-Chess-AI/SP-14-Green-Chess-AI)

Website Link:

Project Website: <https://sp-14-green-chess-ai.github.io/SP-14-Green-Chess-AI/>

Stats and Status:

Number of lines of code: 1,394 lines. (Not including Json data/configuration lines.)

Total man hours:

Estimated: 207

Actual: 188

Status of project: 100% of all functional and nonfunctional requirements are met.

Table of Contents

This document doesn't have any headings. To add headings to your Table of Contents, go to [Home > Styles](#)

1. Project Overview / Abstract (Research):

The Objective task is to build a web-based application of visualize game board for chess. As well as creating a game mode for either player vs player, or ai on the game of chess. The engine is run with a neural engine with emphasis on self-play by deep learning and reinforcement learning via synthetic data of self-generated playing against itself (Alpha-Zero style).

2.Requirement specs:

- Gather and define project functional and non-functional requirements for Software Requirements Specification (SRS) document.
- **Non-Functional Requirements (some examples)**
 - **Maintainability:** The code will use Object-Oriented Programming, modular, and be commented appropriately to keep proper organization.
 - **Compatibility:** The application should be able to run on any operating system with no issues.
 - **Reliability:** The game does not make illegal moves and enforces the rules of the game of chess. Also, the game should not crash from fatal errors.
 - **Usability:** Users should be able to move pieces with ease if it's a legal move. Invalid moves should reset the piece back to its original position, and prompt or indicate to user that the move was invalid.
- **Functional Requirements (some examples)**
 - Game Modes: Player vs Player and Player vs AI
 - Game Rules: The rules of chess must be enforced and follow the world standard rules for the game. Captured pieces must be removed from the board.
 - UI: Allow users to drag and drop pieces when moving a piece. Provide a chess board that saves the current state and updates the board after each valid move.
 - Reset function: Function used to reset the game after a user has finished their match or has made a mistake.

3. Design Document:

- **Class diagram**
 - Chess Board

- Chess Pieces (Pawn, Rooks, knights, Bishops, king and queens)
- Move validator: Checks for legal moves on board
- Game director: Turns, rules, and win or lose conditions
- Ai Engine (Minimax algorithm)
- UI mockup Board and drop pieces

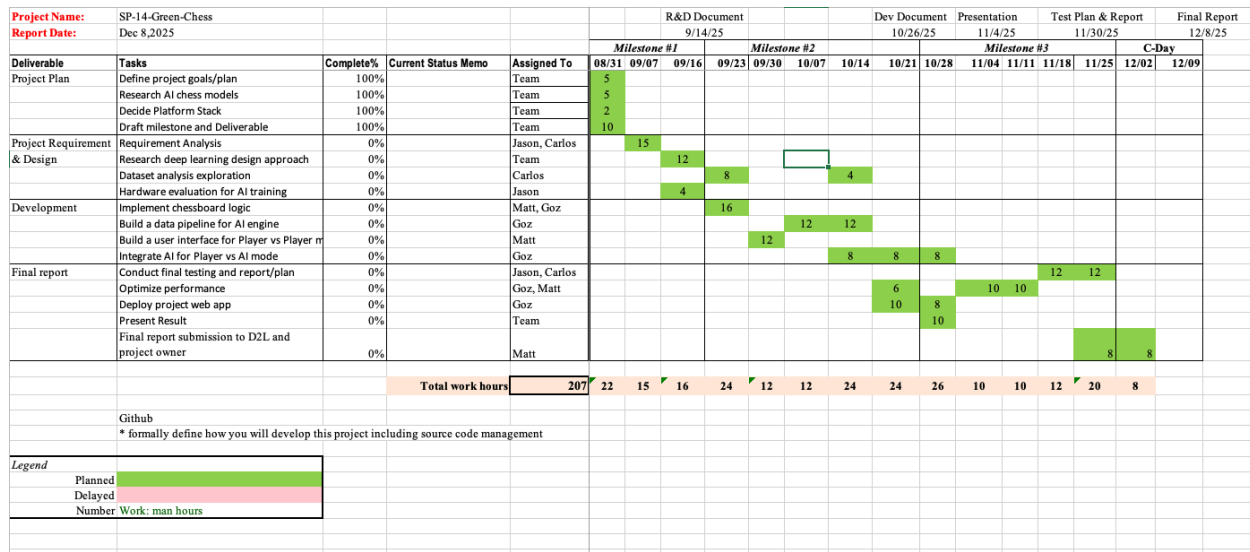
4. Development Document:

- **Project Process**
 - **Project Overview SDLC**
 - Objective and scope of the project
 - Software Development Lifecycle model chosen
 - **Project Plan**
 - Timeline (milestone)
 - Role & Responsibilities
 - Platform & Framework selected
 - **Requirement & Design Documentation**
 - **Requirement**
 - **Functional Requirements**
 - **Non-Functional Requirements**
 - **Design**
 - **Flowchart**
 - **Class diagram**
 - **System architecture diagram**
 - **UI design prototypes**
 - **Development Documentation**
 - Model selection
 - Integration Testing Documentation
 - Testing Documentation
 - Debugging info
- **Source codebase documentation**
 - Using GitHub organization account for version control
 - Add proper code documentation into the source code, for readability and maintainability
- **Final Report/ Research paper**
 - Report on the Final Project Document
 - Project Website
 - Presentation Slide
 - Presentation Demo Video

5. Software Testing Plan:

- **Unit Testing**
 - Test each module individually, their functions, methods, and classes.
 - Verify that the functionality is working as intended by itself
- **Full Integration Testing**
 - Test all modules after they have been merged and how they work with each other.
 - Make sure all parts work together and cause no issues or errors.
- **System Testing**
 - Perform functional tests
 - Performance tests to make sure the system is dependable.
 - Stress tests the program to test the performance of the game under not ideal conditions.

GANTT CHART



1.0 Requirements

1.1 Overview

AI Chess project is a full stack development web-based application that utilizes AI to play against user, or player against player. Using open-source framework of React, web API, chessboard.js, Pytorch, and Python-Chess for development.

1.2 Project Scope

Build a functional web application that allows the users to play chess. The application will offer two different game modes: User vs User and User vs AI. The game will follow all the standard rules of chess. While having an interactive UI to move the pieces around the board allowing users to drag and drop or click and place.

2.1 Environment

The Web based application will be a local environment, where users can play locally against one another or AI engine to play against. Due to time constraints, Later improvements are games can be played over the internet and adding difficulty level of the ai engine.

2.2 System

Utilizing full stack development, there will be a user interface running on a web browser using react library from JavaScript, and back-end development will be built on python and open-source libraries for handling game state, game logic, player moves, and system processing.

3.0 Functional Requirements

3.1 Game Modes

3.1.1 User vs User Mode

- The system will allow two human players to play on the same machine.
- The system should switch between white and black chess pieces depending on whether it's their turn.

3.1.2 User vs AI Mode

- The system will allow human controlled players to participate against AI.
- The AI will generate movement based on the engine, using (Minimax algo, etc.)
- The system should allow the player to pick their colors(optional)

3.2 Game Rules

3.2.1 Standard Rules

- The system will enforce rules based on if the rules are legal in chess, this includes each individual piece movement and capture.

3.2.2 Special Rules

3.2.2.1 Castling

- The system will allow castling depending on if neither king nor rook have moved from their previous position and no check conditions prevent it. No pieces should be blocking king and rook.

3.2.2.2 Pawn Promotion

- The system should allow pawns to be promoted to a queen, rook, knight, or bishop, when they reach the end side of the board.

3.2.2.3 En Passant

- The system should implement En passant capture is when a pawn moves two squares, and an enemy pawn is in the position.

3.2.2.4 Double Step Pawn Move

- The system will allow pawns to move forward two squares from their initial rank

3.3 Database

3.3.1 PGN game state

- The system will store game history in Portable Game Notation format

3.3.2 Player's moves

- The system will log and track each player's moves.

3.3.3 AI engine data

- The system should store the evaluation AI data such as search depth and evaluated positions (optional)

3.3.4 AI generated moves

- The system should log the AI's selected movements for analysis

3.4 UI

3.4.1 Chessboard

3.4.1.1 Drag & drop pieces

- The system should support drag and drop pieces for movement

3.4.1.2 Checkmate, Stalemates Check for win condition

- The system will automatically detect all game ending conditions like checkmate, stalemate to progress games.

3.4.1.3 Player's turn (white or black)

- The system should designate a clear outline to indicate which player's turn it is.

3.4.1.4 Win/loss score counter

- The system should display some sort of scoreboard to show all outcomes and update some player statistics.

3.5 AI Engine

- Min-Max Algorithm
- Machine Learning Training
- Move generation

4.0 Non-Functional Requirements

4.1 Maintainability

The code will use Object-Oriented Programming, modular, and be commented appropriately to keep proper organization.

4.2 Reliability

The game does not make illegal moves and enforces the rules of the game of chess. The game should not encounter any fatal bugs that will crash or disconnect them from their matches.

4.3 Usability

Provide an easy to use and understandable interface for users to navigate. Users should be able to move pieces with ease if it's a legal move. Invalid moves should reset the piece back to its original position, and prompt or indicate to user that the move was

Design Overview

1.2.2 System Overview

- Rules Engine: Chess.js makes sure all moves follow all official chess rules.
- Data layer: In JSON format the moves and game states are stored
- Ai Engine: Uses minimax to evaluate and select moves.
- game director: coordinates rules, AI and database updates.
- Web Interface: Displays the board in chessboard.js and manages the front-end user interaction through React.

Design Considerations

2.1 Assumptions and Dependencies

- The chess game backend will be developed in Python.
- The system will run in a web browser using React to build the UI
- chess.js will handle all the rule validation and the game state management
- Chessboard.js will render the interactive chessboard
- All game data which include moves, states, metadata will be stored in a JSON based database (optional)

2.2 General Constraints

- Platform must run in a desktop or laptop web browser (Chrome, Firefox, Edge, and Safari with proper testing)
- No mobile or tablet devices
- Performance: Ai must compute moves fast enough for smooth gameplay.
- No cloud deployment, multiplayer head-to-head limited to locally only.
- No AI difficulty scaling. AI will make the best optimal choice based on our custom minimax algorithm
- No save and load game functionality. The game must be played and finished in one session.
- Game data will be stored in JSON format

2.3 Development Methods

The chess system will be developed under the Agile framework, combined with CI/CD (Continuous Integration and Continuous Deployment) practices. The Agile framework allows the team to work iteratively and incrementally, breaking the project into separate sprints that focus on specific features such as the frontend chessboard, AI engine functionality, or game logic processing. Continuous feedback from the Agile process provides flexibility for adapting to changing requirements and incorporating validation from user or client input.

To support Agile development, CI/CD practices are incorporated to maintain high code quality throughout the project. Every code change pushed to the repository is built, compiled, tested, and validated before merging into the main branch. This ensures that errors are caught early, reduces the cost and delay associated with unexpected bugs, and preserves overall code quality. The combination of Agile and CI/CD practices enhances the efficiency, reliability, and scalability of the system, ensuring that future releases can be delivered smoothly and effectively.

- Modular coding: Encapsulates each responsibility into its own class or class.
- Scenario Based testing: test all valid conditions through test games.

Architectural Strategies

3.1 System Architecture

The architecture of the system implements strategies of building upon existing software libraries for the development of web-based applications. The frontend utilizes React.js for our user interface, with Chessboard.js for gameboard visualization. The backend stores the game state, logic, and AI engine in Python, using the Flask API to communicate with the frontend and update the chessboard for real-time gameplay against the AI.

This strategy helps with maintainability and scalability by separating the two components, ensuring modularity when developing game logic, AI processing, and the user interface. The clear separation of concerns between the user interface and game logic allows the frontend and backend to operate independently without affecting existing dependencies, enabling scalability to add additional components or features in the future. Other considerations, such as building the system in a single unified language or as a desktop application, would limit flexibility for future enhancements, including web-based multiplayer support and cross-platform compatibility.

3.2 Input and Output Model

The system's input and output model is defined as follows: the user provides input by dragging and dropping a valid chess move, and the output is an updated game state with

the new chessboard rendering. The format follows PGN stored in JSON files, which communicates with the Flask API to update the backend game state, as well as the frontend chessboard. The AI engine then evaluates the game state, generates a move, and updates the game state, which is relayed through the API to update the user interface and reflect the new game state.

3.3 Testing and Quality Assurance

For testing purposes, the system will implement a continuous integration (CI) pipeline to ensure that every build is compiled, validated, and tested before code is pushed to the main branch. This approach allows for early bug detection, smoother integration across components, and the preservation of overall code quality.

Unit testing will validate individual functions and modules, helping to identify data bottlenecks or logic errors at the component level. Black Box testing will ensure that the system behaves correctly from a user perspective, confirming that the application meets functional requirements and delivers a reliable user experience. Integration testing will focus on the communication between the frontend (React.js with Chessboard.js) and the backend (Flask API with Python-based game logic), while end-to-end testing will verify that the entire workflow operates seamlessly. These layered testing approaches enhance maintainability and scalability, ensuring the system remains reliable as new features and components are introduced.

3.4 Development Environment and Version Control

To maintain accessibility and maintainability of the source code, we use GitHub for version control and file sharing throughout the development process. By creating an organizational account, all members have permission to contribute to the codebase, ensuring that the project remains up to date, supports efficient collaboration, and provides a structured environment for maintenance and organization. Our preferred IDE is Visual Studio Code, chosen for its versatility and ease of use in web development. Its wide range of extensions, built-in Git integration, and support for multiple programming languages make it well-suited for managing both the frontend and backend components of the project.

System Architecture

This section presents a high-level architectural overview of the Chess AI Game Website and the logical partitioning of its responsibilities among subsystems and major components. The design supports modularity, agile workflows, and ease of modification, in line with a scenario-based approach.

Functional Partitioning

- **Frontend:** The user interface is built with React. All interactive elements, game state presentations, and user input handling occur on this layer. The frontend communicates with the backend using JSON over HTTP (WebSocket for real-time updates), and employs chess.js to execute move validation, legal move generation, and lightweight local game state management.
- **Backend:** Implemented in Python, the backend provides core application logic, game management, persistent storage, AI computation, and orchestrates communication among modules. The backend exposes a REST API, returning and accepting JSON payloads for all data exchange.
- **AI Engine:** A modular component within the backend, implemented in Python, responsible for computing moves using configurable AI logic. This component is designed for extensibility and scenario-based testing (swapping algorithms or difficulty levels).
- **Storage/Database Subsystem:** Responsible for saving user data, game histories, and scenario configurations. For local operation, this may be a lightweight file-based solution (JSON files).
- **Scenario Module:** Encapsulates scenario-based workflows, such as tutorials, puzzles, and analysis. This module leverages the same move-processing infrastructure as regular gameplay but loads and manages custom board states and solution paths.

- Integration Layer: Handles communication protocols and data serialization/deserialization between frontend, backend, and scenario modules, ensuring all interactions happen via strict JSON schemas.

4.2 Subsystem Architecture

Each subsystem is independently developed and tested, allowing for modular upgrade and scenario-based extensions such as puzzles, analysis, or new AI difficulty settings. The architecture supports agile workflows, with clear API boundaries enabling parallel tasking for frontend and backend teams.

4.2.1 Frontend (React)

- Implements the user interface, including the chessboard, controls, move history, and settings.
- Communicates with backend APIs via asynchronous JSON requests
- Receives real-time game updates (moves, AI responses, game state) through HTTP or Web Sockets.
- Utilizes chess.js locally for immediate client-side move validation, allowing for smooth UI feedback and optimistic updates.
- Modular components (board, move list, chat, scenario selector) encourage reuse and isolated testing.

4.2.2 Game Logic/ AI Engine (Python and chess.js)

- Python provides core AI logic (move calculation, difficulty modes, scenario-aware play, logging).
- chess.js is used for legal move validation, FEN parsing, and core chess rules enforcement.
- Scenario-based build tools (Python-driven or config-file-based) inject custom game setups, AI constraints, or puzzle definitions via JSON.
- RESTful APIs expose endpoints for move submission, board state retrieval, analysis queries, and user statistics.

- Modular design allows pluggable AI backends, new scenario engines, or third-party chess services to be integrated or swapped as independent services.

4.2.3 Data/API Layer

- JSON serves as a universal serialization format for all API payloads, game states, and configuration data.
- WebSocket or HTTP endpoints encapsulate stateless and stateful interactions, supporting both turn-based and live (real-time) play.
- Persistence (DB or in-memory cache) is abstracted behind clear interfaces, allowing for easy swap or upgrade without affecting API clients.
- All scenario data (puzzles, training modules, variant rules) is stored and transferred as JSON objects, supporting easy import, versioning, and extension.

Detailed System Design

5.1 Classification

- Game Manager Module
- AI Engine Module
- UI Module
- State Management Module
- Utilities Module (Validation, Logging)

5.2 Definition

- Game Manager Module: Orchestrates chess gameplay, managing turns, player input, and interactions between the AI and UI modules. It also maintains the core game state and communicates with scenario loading.
- AI Engine Module: Implements the chess AI, featuring both classic search (Minimax with Alpha-Beta pruning) and extensible machine learning-

based evaluation. It determines AI moves and supports multiple difficulty levels and scenario-based constraints.

- UI Module: Handles all user interactions, board rendering, move input, and displays results. Designed for fast local feedback and easy resetting or testing of scenarios.
- State Management Module: Maintains game state in-memory, serializes positions as JSON/FEN, and supports undo/redo and scenario reloading.
- Scenario Loader Module: Loads and validates custom start positions, AI/game constraints, or training scenarios from local files (JSON or FEN). Supports rapid scenario switching for AI training or testing.
- Utilities Module: Provides chess move validation (chess.js or equivalent), logging, error handling, and configuration management services.

5.3 Constraints

- Designed for execution on typical local environments (laptops/desktops) running Python 3.8+ and supporting chess.js (JS validation/UI).
- Must perform state transitions and move validation atomically to prevent desynchronization, especially during rapid scenario testing or self-play loops.
- AI Engine must support switching evaluation functions to allow experimentation with both rule-based and machine learning-driven move selection.
- All components must permit stateless resets to support agile training and scenario exploration.
- Memory usage should remain efficient, supporting at least thousands of moves/states in RAM for analysis/training.

5.4 Resources

Hardware Resources:

- CPU processor: to execute the game logic, UI, AI computation moves
- RAM memory: To store the program instruction, game state, and PGN history
- Display: a monitor that displays the Chessboard UI inside a web browser

- Input: Mouse to drag and drop chess pieces and interact with the UI

Software Resources:

- Chessboard.js - UI chessboard visualization
- Chess.js - JavaScript Library for handling all game logic, with move validation and update game state.
- React Framework- Open-source JavaScript frameworks for building User Interfaces
- Flask API- python web application framework for server-side logic

Data Storage: all information will be transmitted to PGN/JSON format to store game state, game history (AI to generate game moves).

1. Test Strategy

4.1 Manual Functional Testing

The team will be testing the web application directly to validate the application's functionality and its correctness. Including things like the UI, the four game modes, and AI move generation and rule enforcement.

4.2 Integration Testing

Our team will test the interactions between all game engines, and all three AI engines, and the multiplayer system. Testing to make sure the game state is persistent, that is synchronized and working, and each integration is working correctly across all the components of the application.

4.3 Regression Testing

The team will test after each update and feature that is added. Using back testing to test that each past update still works, and previous features remain unaffected by the new feature. Especially, focusing on the key functionalities is still intact and functional.

4.4 Test Case Prioritization

The team will focus on test cases with high or very high priority as these relate to functional requirements. Things like core game play rules, end game detection, and AI move generation. Cases will be tested manually in certain scenarios, and the tester will document and observe the results.

2. Test Cases

5.1 Local 2 Player Mode Test Cases

Test ID	Test Objective	Action / Procedure	Expected Result	Actual Result	Priority	Pass/Fail
LM01	Valid piece move	- Try all different pieces for valid piece placements.	Every piece should follow their movement rules and only move to valid spots.	Valid moves were all accepted for every chess piece.	Very High	Pass
LM02	Illegal move rejection	- Try illegal move	The piece will be moved back to its current position, rejecting the invalid move.	Move was rejected and the piece was not moved.	Very High	Pass
LM03	Turn validation	- try to move black out of turn	White and black pieces should	Turn validation is correct.		

		- try to move white out of turn	only move when it's their turn.			
LM04	Checkmate detection	-play game till checkmate position	The game ends and the board is now locked.	Game ends and board is locked.	Very High	Pass
LM05	Stalemate detection	-play till stalemate position	The game ends and the board is now locked.	Game ends and board is locked.	Medium	Pass
LM06	Check detection	-play till check position	Game is locked till a move, moves the user out of check.	Players have to make valid moves out of check.	Very High	Pass
LM07	Special rules work and valid (castling, double pawn step, etc.)	-play till all special rules	Castling when valid, double pawn step and en passant when valid.	Special rules all work in valid situations.	High	Pass
LM08	Pawn promotion	-play till pawn promotion	A pawn should be promoted to the queen.	The pawn is correctly promoted.	High	Pass
LM09	Reset game control	-make a move - hit reset button	The board should reset back to the start.	Chessboard correctly resets.	High	Pass
LM010	Undo move control	-make a move - hit undo button	The last move made should be reset back to the previous position, and the history panel updated.	The last piece is correctly reset, and the history panel is updated.	Medium	Pass
LM11	Flip Board	-hit the flip board button	The board should switch sides.	The board correctly flips positions.	Low	Pass

			Moving the black pieces to bottom and white to the top. Or vice versa.			
--	--	--	--	--	--	--

5.2 Player vs AI Mode Test Cases

Test ID	Test Objective	Action / Procedure	Expected Result	Actual Result	Priority	Pass/Fail
AI01	Stockfish engine piece movement	-start player vs ai -select stockfish -hit make engine move button	Stockfish engine makes a valid move.	Stockfish makes a valid move.	Very High	Pass
AI02	Leela engine piece movement	-start player vs ai -select Leela engine -hit make engine move button	The Leela engine makes a valid move.	Leela makes a valid move.	Very High	Pass
AI03	Minimax engine piece movement	-start player vs ai -select Minimax engine -hit make engine move button	Our custom minimax engine makes a valid move.	Custom minimax makes a valid move.	Very High	Pass
AI04	Evaluation bar updates per move	-start player vs ai - make a move -make an engine move	Evaluation score should display per move, user or ai. And display a score.	A score is displayed on the last move made.	Medium	Pass

AI05	AI engine selection control	-start player vs ai -select different ai engines	Pick any engine from dropdown. And make the engine move.	You can select any AI engine from the list and make the engine move at the users' will.	High	Pass
AI06	AI engine turn control	-Check and make sure AI only moves when button is pressed	AI should not move unless a button is clicked.	AI does not move until the button is pressed.	High	Pass
AI07	Legal move validation	-start player vs ai - check console for illegal moves trying to be made by AI.	AI should not try to move illegally, if so, it's caught and new move is produced.	AI only makes valid moves for its turn.	High	Pass

5.3 Multiplayer Online Mode Test Cases

Test ID	Test Objective	Action / Procedure	Expected Result	Actual Result	Priority	Pass/Fail
MM01	Create a new game with UUID	-select multiplayer game mode - create a new game room.	A new room should be created and set up with a unique generated ID for the room.	A new room was created with a unique UUID.	Medium	Pass
MM02	Join game from lobby list	-select multiplayer mode - click drop down lobby list - select one from the list	The user joins a game room with an open spot.	User successful can join rooms.	Medium	Pass
MM03	Join game with lobby id	-enter lobby id in text field asking for id	Users should be brought to the correct game room	The user is correctly brought to the right room.	Medium	Pass

			with the correct code.			
MM04	Available list of open games in lobby	-select multiplayer mode -select drop down list to view open game rooms.	User clicks drop down of rooms; list is displayed with only games with an open spot.	List is displayed, and full games are hidden.	Medium	Pass
MM05	Player white move sync	-start a game, allow second player to join - start playing -monitor game state, turn validation	Player white move should display on player black's screen and be in sync.	The move player's move shows up on the opponent's screen and is up to date.	High	Pass
MM06	Player black move sync	-start a game, allow second player to join - start playing -monitor game state, turn validation	Player black move should be displayed on player white's screen and in sync.	The player's move shows up on the opponent's screen and is up to date.	High	Pass
MM07	Game State persistency	-start a multiplayer game - finish the game	The game should stay consistent with both users making moves.	The game stays consistent and up to date.	High	Pass
MM08	Reconnect to lobby handling	-have a multiplayer game going -leave and try to reconnect to the same game room.	Users should be able to rejoin the current game room they were in before disconnecting.	Users can reconnect, but the game state is not consistent.	Low	Fail
MM09	Chat messages send and receive	-start or join a multiplayer game	Both users should be able to send and receive messages	Chat message back and forth worked for	Low	Pass

		- both users send message to test chat	with the correct timestamps	both users in a game room.		
MM10	Correct turn reinforcement	-start a multiplayer game - make sure white and black can only move on correct turn	Players playing white and black should only be able to move when it's their turn.	Turn validation is enforced for both users.	High	Pass

5.4 AI vs AI Mode Test Cases

Test ID	Test Objective	Action / Procedure	Expected Result	Actual Result	Priority	Pass/Fail
AA01	Automated game simulation	-select engines - click start ai vs ai button	Engines take alternate turns making moves automatically.	Engines make moves and play.	High	Pass
AA02	Stop Functionality	-start ai vs ai match - hit the stop button	The match should stop, and the board will reset.	The match stops and the board is reset.	High	Pass
AA03	Play to end game				Medium	Pass
AA04	Update Move History	-start ai vs ai - Watch the move history panel for updates.	History should display the complete turn for white and black.	The history panel displays the move made by AIs.	Low	Pass
AA05	Same three moves in a row force draw scenario	-start ai vs ai - watch move history for three move repeats & game stoppage -repeat if the case is not triggered.	When the same three moves are made in a row by one player, the match should end in a draw.	Match stops and ends in a draw.	Medium	Pass

AA06	Same move sequence three times in a row force draw scenario	-start ai vs ai - watch move history for three sequence repeats. -repeat if the case is not triggered.	When the same move sequence is made by both AIs in row three times. The match should end in a draw.	Match stops and ends in a draw.	Medium	Pass
AA07	Not enough pieces left for capture, force draw scenario	-start ai vs ai - watch games and check console logs for a draw. -repeat if the case is not triggered.	The match should stop and end in a draw, when both players do not have enough pieces to checkmate.	The match stops and ends in a draw.	Medium	Pass
AA08	Same AI engine vs itself	-select the same engine for white and black -start ai vs ai match	The match simulation begins with the AI vs itself	The match starts and it begins playing itself in a game. Any of the 3 engines.	Medium	Pass

5.5 Game State and Persistence Test Cases

Test ID	Test Objective	Action / Procedure	Expected Result	Actual Result	Priority	Pass/Fail
GP01	Game status tracking	-play a game till check or checkmate -check status	Status of game is updated correctly	Status is updated	Very High	Pass
GP02	JSON save and update per move	-make a valid move - check saved	JSON file is created (if first move) or updated	JSON file is created/updated	Medium	Pass

		folder in project				
GP03	Chat persistence	-send message -reload multiplayer game	Chat history has been restored.	Chat history is not restored.	Low	Fail
GP04	Load saved game state	-close app -restart -load game	Game resumes from saved position.	The board is not resumed on rejoining player	Medium	Fail

5.6 UI Customization Test Cases

Test ID	Test Objective	Action / Procedure	Expected Result	Actual Result	Priority	Pass/Fail
UI01	Changing piece theme color	-select a game mode - change piece theme to a different color	The chess pieces will change for the correct color styles. Should work and be applied to them all.	Every color piece style worked and changed to the correct color patterns selected.	Low	Pass
UI02	Changing board theme color	-select a game mode - change board theme to a different theme.	The board color scheme should change to the selected option. Should work and be applied to them all.	The board correctly changed to the correct theme depending on the option chosen.	Low	Pass
UI03	Toggle different piece styles	-select a game mode - change the piece style to a different style.	The pieces should change to a different style. Currently offering 2.	The chess pieces all changed to whatever style was selected.	Low	Pass
UI04	Move History	-start a game - move pieces and	The history panel should display the	History is displayed correctly and in the	Low	Pass

		do multiple complete turns for both sides.	moves in the format of complete turns. White and black piece moves will appear in the same line for a turn.	right format.		
UI05	Chat timestamps	-start a multiplayer game. - wait for player 2 to join the lobby. -send messages in chat.	The timestamp for each received message should have the correct time.	Each message displays a timestamp underneath.	Low	Pass

6. Test Procedure

6.1. Local Mode

6.1.1 Legal Moves

- Step 1: Drag a chess piece with the mouse
- Step 2: System checks if the move is legal
- Expected Result: Chessboard updates to reflect the legal move

6.1.2 Illegal Moves

- Step 1: Attempt to play a move out of turn or an illegal move
- Step 2: System checks if the move is illegal
- Expected Result: Board does not update; move is rejected

6.2. Player vs AI

6.2.1 AI Moves

- Step 1: Select Player vs AI mode
- Step 2: Select an AI engine (Stockfish, Leela, Minimax, etc.)
- Step 3: Click the button to generate AI moves
- Expected Result: AI move is applied to the client-side gameboard

6.3. AI vs AI

6.3.1 AI Selection

- Step 1: Select AI vs AI mode
- Step 2: Choose engines for both sides
- Step 3: Start AI vs AI mode
- Expected Result: Each AI generates moves on its respective turn

6.3.2 Game Termination

- Step 1: Allow game to reach a draw or checkmate state
- Expected Result: Game ends and winner/draw is declared

6.4. Multiplayer

6.4.1 Create a Room

- Step 1: Select Multiplayer mode
- Step 2: Create a room
- Step 3: Join as Player 1
- Expected Result: WebSocket connection established to a new saved gameboard on the backend server

6.4.2 Join a Room

- Step 1: Select Multiplayer mode
- Step 2: Enter or select an existing room ID
- Step 3: Join as Player 2
- Expected Result: WebSocket connection established to an existing saved gameboard on the backend server

6.4.3 Chat

- Step 1: Join a room
- Step 2: Enter a name and submit a chat message
- Expected Result: Message is broadcast to all players and saved to the gameboard room's chat history in the backend server

6.5. UI Customization

6.5.1 Customizing Gameboard

- Step 1: Select options such as flip board, board color, or piece style

- Expected Result: Gameboard updates to display selected customization on the client UI

6.5.2 UI interface

- **Step 1:** Click on the website
Step 2: Verify the display of:
 - o Game modes
 - o Chessboard UI
 - o Move history
 - o Customization options
 - o Reset / Undo moves features
- **Expected Result:**
 - o Website renders correctly.
 - o All UI elements are functional.
 - o Users can select and play a chess game across different modes.

7. Test Environment

7.1 Software Requirements

Version Control:

- GitHub – Remote repository for storing and managing source code versions.

Web Browsers:

- Google Chrome / Chromium
- Mozilla Firefox

Development Environment:

- IDE or code editor with support for React JavaScript and Python (e.g., VS Code, PyCharm).

Frontend Packages (React/JavaScript):

- react – Core frontend framework
- react-chessboard – UI component for rendering the chessboard
- chess.js – Library for chess logic and move validation

Backend Packages (Python):

- fastapi – Framework for backend API services
- uvicorn – ASGI server for running FastAPI
- python-chess – Chess logic and AI integration
- websockets – Real-time communication between client and server

7.2 Hardware Requirements

Internet Connectivity:

- Required for public/global accessibility, multiplayer connections, and hosting services.

Server Hosting Services:

- Static hosting: GitHub Pages (for frontend deployment)
- Cloud hosting providers for backend APIs (e.g., Render, AWS, Azure)

Local Test Environment:

- Local machine running on Windows or macOS
- Capable of running local development servers (React and FastAPI) and supporting required libraries

8. Test Data

Data requirement

- **File Format:**

All test data files must be in **JSON format** to ensure compatibility with the system's data parser and backend services.

- **User Move Storage:**

The data must store each user's chess to move input and accurately reflect **move history information** for all game modes.

- **Message Differentiation:**

The data layer must distinguish between different **data message types** (e.g., move input, game state update, AI response, chat messages) and process them correctly across multiple game modes.

- **Client Identification:**

Each data message must include a **client ID** to uniquely identify HTTPS requests from different users.

- **Game State Management:**

The system must store the **game state** for each client across all game modes (e.g., AI vs player, AI vs AI, online multiplayer).

- **AI Move Integration:**
The data must support loading and generating **AI moves** from compatible Chess AI engines and store this information as part of the game data.
- **Server Connectivity:**
The test data must support establishing and validating **web connections** between the client and server backend.
- **Frontend–Backend Data Exchange:**
Data must be retrievable from the backend and processed by the frontend, including real-time game state updates, session data, and validation results.
- **Real-Time UI Updates:**
Data processing must enable real-time updates to the **user interface**, such as animations, and chat activity.

9. Assumptions and Dependencies

9.1 Assumptions

- Users have knowledge of how to play the game chess and understand the rules.
- The user's browsers are up to date and properly installed.
- The Leela engine is available to use locally.
- All required dependencies are installed in their project setup (npm, pip, chess.js packages).
- The backend server is running and is accessible to users.
- Users know how to operate and use the UI.

9.2 Dependencies

- **Team Coordination:** Multiple team members needed to be on to play and test multiplayer capabilities.
- **Network Connectivity:** Testing the multiplayer requires network connectivity to test if WebSocket connections are functional.
- **AI Engines:** Leela and Stockfish require proper installation and setup.
- **Test Data:** Saved game JSON files must be accessible.
- **Backend Availability:** FastAPI server needs to be running, or testing cannot occur for the web application.

3.4 Development Environment and Version Control

To maintain accessibility and maintainability of the source code, we use GitHub for version control and file sharing throughout the development process. By creating an organizational account, all members have permission to contribute to the codebase, ensuring that the project remains up to date, supports efficient collaboration, and provides a structured environment for maintenance and organization. Our preferred IDE is Visual Studio Code, chosen for its versatility and ease of use in web development. Its wide range of extensions, built-in Git integration, and support for multiple programming languages make it well-suited for managing both the front-end and back-end components of the project.

Version Control Plan:

Our project code will be hosted on GitHub, and we will use Git repository for user collaboration and version control. This will allow us to track code changes and to revert to previous versions. Allowing us to collaborate more efficiently with code reviews on pull requests before merging the code into production.

Basic UML Diagram of the System Architecturewqsa